

Analisi Dettagliata: Dinamiche di Caccia in "Simbiosi Silente"

Ecco una scomposizione passo-passo di come funziona un inseguimento, basata sulle regole matematiche del codice.

1. L'Inizio: La Scoperta e la Fuga

La caccia non inizia finché due condizioni non sono soddisfatte:

1. Il Predatore (Cacciatore):

- Deve essere pronto a cacciare: non deve essere in "STOP" (!isResting) e non deve essere esausto (fatigue < 70).
- Trova una preda all'interno del suo raggio di percezione (genetics.sense).
- **Condizione Chiave:** Il predatore deve essere *più grande* della preda (this.currentSize > altra.currentSize).
- Se queste condizioni sono vere, imposta la preda come this.targetCreatura (Stato: "IN CACCIA").

2. La Preda (Produttore):

- Se un predatore (che non è in "STOP") entra nel suo raggio di percezione (genetics.sense)...
- ...e la preda non è essa stessa in "STOP"...
- Imposta il predatore come this.fuggendoDa (Stato: "IN FUGA").

2. La Corsa: Velocità vs. Fatica

Questo è il punto cruciale. La velocità massima è genetica, ma la velocità *effettiva* dipende dalla fatica (fatigue).

Formula Velocità Effettiva (per Entrambi):

- fatigueFactor = 1 - (fatigue / 100) * 0.5
 - *Spiegazione:* La fatica può ridurre la velocità massima fino al 50%. Se fatigue è 100, il fatigueFactor è 0.5.
- effectiveSpeed = genetics.speed * fatigueFactor

Chi Rallenta Prima? L'Accumulo di Fatica

La preda si stanca molto più velocemente del predatore.

- **Preda (IN FUGA):**
 - Accumulo: fatigue += 1.0 (ogni frame)
 - Variabile: FATIGUE_FLEE_RATE
- **Predatore (IN CACCIA):**
 - Accumulo: fatigue += 0.3 (ogni frame)

- Variabile: FATIGUE_HUNT_RATE

Analisi: La preda accumula fatica circa 3 volte più velocemente del predatore. Questo dà al predatore un vantaggio di resistenza: se l'inseguimento dura abbastanza, la preda diventerà inevitabilmente più lenta del predatore, anche se la sua velocità genetica di base era superiore.

3. La Fine: Esaurimento o Cattura

Un inseguimento termina in tre modi:

A. Esaurimento Energia (Lo "STOP")

Entrambe le creature consumano energia (`vivi()`) ad ogni frame, ma il predatore paga un costo metabolico più alto (`PREDATOR_COST_MULTIPLIER = 1.2`).

- **Condizione:** energia < 30 (`ENERGY_REST_THRESHOLD`)
- **Azione:** La creatura entra in `isResting = true` (Stato: "STOP").
- **Recupero:** L'energia si ricarica passivamente a 0.15 per frame.
- **Ripresa:** Il movimento riprende quando energia >= 50.
- **Scenario:** Se il predatore esaurisce l'energia prima di raggiungere la preda, la preda fugge. Se la preda si ferma per prima, diventa un bersaglio facile.

B. Esaurimento Fatica (Solo Predatore)

Se la preda è troppo veloce o distante:

- **Condizione:** fatigue > 70 (`FATIGUE_HUNT_THRESHOLD`)
- **Azione:** Il predatore smette di cercare nuove prede (`targetCreatura = null`) e passa allo stato "CERCA CIBO" (`pellet`) per recuperare fatica (fatigue -= 3.0 per frame).

C. La Cattura (Successo)

- **Condizione 1 (Distanza):** distanza < (`predatore.currentSize * 2.0`)
- **Condizione 2 (Dominanza):** `predatore.currentSize > preda.currentSize`
- **Azione:** La preda viene uccisa (`altra.energia = 0`), il predatore guadagna energia (+ `altra.energia * 0.9`), recupera fatica (fatigue -= 20) e cresce (`currentSize += 0.2` se l'energia è piena).